

LeetCode 202: Happy Number

Problem Analysis & Multi-Approach Solutions

JPMC

TCS

Swiggy

Zoho

Author: Arpit Raj
Institution: LNMIIT Jaipur
Date: June 24, 2026
Ref: Q. 154 / LC 202

Conceptual Walkthrough & Examples

A happy number is defined by a process where a number is replaced by the sum of the squares of its digits repeatedly until the number equals 1, or it loops endlessly in a cycle which does not include 1.

EXAMPLE 1: HAPPY NUMBER (N = 19)

$19 \rightarrow (1)^2 + (9)^2 = 82$
 $82 \rightarrow (8)^2 + (2)^2 = 68$
 $68 \rightarrow (6)^2 + (8)^2 = 100$
 $100 \rightarrow (1)^2 + (0)^2 + (0)^2 = 1 \rightarrow \text{TRUE}$

EXAMPLE 2: UNHAPPY NUMBER / CYCLE DETECTED (N = 2)

$2 \rightarrow 4 \rightarrow 16 \rightarrow 37 \rightarrow 58 \rightarrow 89 \rightarrow 145 \rightarrow 42 \rightarrow 20 \rightarrow 4$
[Cycle Repeated: The sequence loops infinitely back to 4 without ever reaching 1]

Approach 1: Floyd's Algorithm (Slow & Fast Pointers)

This approach models the sequence as a linked list structure where cycles can be detected using two pointers moving at different speeds.

```
int slow = n;
int fast = n;
while(true){
    slow = nextnum(slow);
    fast = nextnum(fast);
    fast = nextnum(fast);
    if (fast == 1) return true;
    if (slow == fast) break;
}
return false;
```

TIME COMPLEXITY **$O(\log(n))$**

States become very small rapidly.

SPACE COMPLEXITY **$O(1)$**

Constant space usage without tracking elements.

Approach 2: Hashset

Utilizes a hashset to keep track of all previously visited numbers. If a number repeats during the sequence transition, a cycle is confirmed, indicating that the input is not a happy number.

```
unordered_set<int> seen;
while (n != 1) {
    if (seen.count(n))
        return false;
    seen.insert(n);
    n = nextNumber(n);
}
return true;
```

TIME COMPLEXITY **$O(\log(n))$**

Proportional to the length of the chain before terminal behavior.

SPACE COMPLEXITY **$O(\log(n))$**

Memory scales with the number of unique elements encountered.

Approach 3: Hardcoded Cycle Detection (Observation)

OBSERVATION

Every unhappy number eventually enters 4 to meet an endless cycle. Hence, we can specifically check for this state to exit early.

```
while (n != 1 && n != 4) {
    n = nextNum(n);
}
return n == 1;
```

TIME COMPLEXITY **$O(\log(n))$**

Terminates efficiently upon hit.

SPACE COMPLEXITY **$O(1)$**

Zero overhead, completely iterative logic.